

Data Science, Machine Learning

Bienvenue dans ce parcours pratique ! Ce document est conçu pour vous accompagner progressivement dans la découverte de la science des données et du machine learning. Chaque exercice s'appuie sur le précédent pour construire vos compétences pas à pas.

Objectifs du parcours

À la fin de ce parcours, vous serez capable de :

Mener un projet complet

Mener un projet complet de science des données de A à Z

Construire des modèles

Construire et évaluer des modèles prédictifs

Corriger les biais

Identifier et corriger les biais dans vos modèles

Appliquer le RL

Appliquer l'apprentissage par renforcement à des problèmes concrets



Partie I – Votre premier projet Data Science

Pourquoi commencer par un projet complet ?

Plutôt que d'apprendre des concepts isolés, vous allez découvrir la démarche complète en résolvant un problème réel. C'est comme apprendre à cuisiner en préparant un plat entier plutôt qu'en étudiant chaque technique séparément.

- **Marketing** : Pourquoi nos clients partent-ils ? (prédiction du churn)
- **Finance** : Comment détecter les transactions frauduleuses ?
- **Santé** : Peut-on prédire le risque de maladie cardiovasculaire ?
- **E-commerce** : Quels produits recommander à chaque client ?

Consigne pratique :

1. Choisissez un jeu de données sur [UCI Machine Learning Repository](#) ou [Kaggle](#)
2. Rédigez en 5-10 lignes :
 - Le contexte professionnel
 - Le problème à résoudre
 - Les critères de succès (ex : "réduire le taux d'attrition de 15%")



Astuce : Choisissez un domaine que vous connaissez un peu, cela facilitera l'interprétation des résultats.

Étape 2 : Explorer vos données

Objectif : Faire connaissance avec vos données avant de les analyser.

Ce que vous allez faire :

1. Chargement et première inspection

```
# Exemple de code
import pandas as pd
data = pd.read_csv('votre_fichier.csv')

# Premières questions à se poser :
print(data.shape)      # Combien de lignes et colonnes ?
print(data.info())      # Quels types de données ?
print(data.describe())  # Quelles sont les statistiques de base ?
```

- Y a-t-il des valeurs manquantes ? Combien ?
- Y a-t-il des valeurs aberrantes (outliers) ?

3. Visualisations à créer :

- Histogrammes pour voir la distribution des variables numériques
- Diagrammes en barres pour les variables catégorielles
- Matrice de corrélation pour identifier les relations entre variables
- Boxplots pour repérer les valeurs aberrantes

  **Conseil :** Passez du temps sur cette étape ! Une bonne exploration évite des erreurs coûteuses plus tard.

Étape 3 : Nettoyer et préparer vos données

Pourquoi c'est important : Les modèles sont sensibles à la qualité des données. "Garbage in, garbage out" !

Ce que vous allez faire :

01

Gérer les valeurs manquantes

- Option A : Supprimer les lignes (si peu de données manquantes)
- Option B : Imputer (remplacer par la moyenne, la médiane ou une valeur calculée)
- Documentez votre choix et son impact

03

Sélectionner les variables pertinentes

02

Traiter les valeurs aberrantes

- Les identifier visuellement (boxplots)
- Décider : erreur de saisie ? Valeur extrême mais valide ?
- Supprimer, transformer ou garder selon le contexte

04

Transformer si nécessaire

- Éviter les corrélations trop fortes (multicolinéarité)



À documenter : Pour chaque décision, notez pourquoi vous l'avez prise. Cela sera utile pour votre rapport final.

Étape 4 : Comprendre les méthodologies

Objectif : Situer votre travail dans un cadre méthodologique reconnu.

Trois grandes méthodologies structurent les projets data science :

1	2	3
KDD (Knowledge Discovery in Databases) Sélection → Prétraitement → Transformation → Data Mining → Interprétation Accent sur l'extraction de connaissances	SEMMA (Sample, Explore, Modify, Model, Assess) Échantillonnage → Exploration → Modification → Modélisation → Évaluation Approche très technique développée par SAS	CRISP-DM (Cross-Industry Standard Process for Data Mining) Compréhension métier → Compréhension données → Préparation → Modélisation → Évaluation → Déploiement La plus utilisée en entreprise car elle intègre la dimension business

Exercice pratique : Créez un tableau avec 3 colonnes (KDD, SEMMA, CRISP-DM) et listez vos actions dans chaque phase. Par exemple :

- Votre étape 1 correspond à "Business Understanding" (CRISP-DM)
- Votre étape 2 correspond à "Data Understanding" (CRISP-DM) et "Explore" (SEMMA)



Question de réflexion : Quelle méthodologie correspond le mieux à votre façon de travailler ? Pourquoi ?

Livrable de la Partie I

Format : Rapport de 5-8 pages (ou notebook Jupyter commenté)

Contenu attendu :

1 Introduction

Contexte, problème, objectifs

2 Exploration

5-6 visualisations clés avec interprétation

3 Prétraitement

Tableau récapitulatif des décisions prises

4 Méthodologie

Tableau comparatif et justification de votre approche

5 Conclusion

Principaux enseignements et prochaines étapes

Partie II – Machine Learning en pratique

Module 1 : Comprendre le cycle de vie d'un projet ML

Activité de réflexion :

Reprenez votre projet de la Partie I et identifiez :

1. Les acteurs impliqués à chaque étape :

- Qui définit le problème métier ? (Chef de projet, expert métier)
- Qui collecte les données ? (Data engineer)
- Qui construit le modèle ? (Data scientist)
- Qui valide les résultats ? (Expert métier + Data scientist)

2. Les livrables produits :

- Cahier des charges
- Rapport d'exploration
- Données nettoyées
- Modèle entraîné
- Documentation du modèle



Insight : Un projet ML réussi nécessite une collaboration étroite entre profils techniques et métier !

Module 2 : Data Science vs Machine Learning

Tâche	Data Science	Machine Learning
Nettoyer les données	✓	
Calculer des statistiques descriptives	✓	
Visualiser les tendances	✓	
Construire un modèle prédictif		✓
Optimiser les hyperparamètres		✓

Question clé : Quand la science des données suffit-elle ? Quand le ML devient-il nécessaire ?

Réponse :

- Data Science seule = décrire, comprendre, explorer le passé
- Data Science + ML = prédire, optimiser, anticiper le futur

Module 3 : Votre premier modèle prédictif

Objectif : Construire deux types de modèles de base.

A) Régression linéaire (prédire une valeur numérique)

Exemple concret : Prédire le prix d'une maison selon sa surface, nombre de pièces, etc.

Étapes détaillées :

1. Préparer les données


```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)\n)\n
```

2. Entraîner le modèle

```
from sklearn.linear_model import LinearRegression\nfrom sklearn.metrics import mean_squared_error\n\nmodel = LinearRegression()\nmodel.fit(X_train, y_train)\n\n# Prédire sur le jeu de test\ny_pred = model.predict(X_test)\n\n# Évaluer\nmse = mean_squared_error(y_test, y_pred)\nprint(f"Erreur quadratique moyenne : {mse}")
```

3. Interpréter

- Quelles variables ont le plus d'impact ? (regarder les coefficients)
- Le modèle prédit-il bien ? (comparer MSE sur train vs test)

📌 **Signe de succès :** Si votre MSE sur le test est proche de celle sur le train, votre modèle généralise bien !

B) Classification KNN (prédire une catégorie)

Exemple concret : Prédire si un client va se désabonner (oui/non).

Comprendre KNN en 3 étapes :

01	02	03
Le principe	Choisir k (l'hyperparamètre crucial)	Visualiser les résultats

est oui

```
# Tester différentes valeurs de k
k_values = [3, 5, 7, 9, 11, 15, 21]
scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    # Validation croisée 5-fold
    score = cross_val_score(knn, X_train, y_train,
                             cv=5).mean()
    scores.append(score)
    print(f"k={k}: score moyen = {score:.3f}")
```

⚠ Attention :

- k trop petit (ex: k=1) → sensible au bruit (surapprentissage)
- k trop grand → prédictions trop générales (sous-apprentissage)

Métrique de distance : Testez Euclidienne vs Manhattan et comparez !

Module 4 : Clustering - Découvrir des groupes cachés

Cas d'usage : Segmenter vos clients sans savoir à l'avance combien de segments existent.

A) K-means - L'algorithme de base

Le principe illustré :



Gamma fonctionne mieux sur Chrome.

Pour bénéficier d'une expérience optimale, ouvrez cette page dans Chrome.

Implémentation guidée :

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Tester différentes valeurs de k
inertias = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertias.append(kmeans.inertia_)

# Méthode du coude
plt.plot(k_range, inertias, marker='o')
plt.xlabel('Nombre de clusters (k)')
plt.ylabel('Inertie (somme des distances intra-cluster)')
plt.title('Méthode du coude')
plt.show()
```

📄 💡 **Interpréter le coude :** Cherchez le point où la courbe forme un "coude" – c'est souvent le nombre optimal de clusters.

B) K-medoids (PAM) - Plus robuste aux outliers

Différence clé :

K-means

K-medoids

```
from sklearn_extra.cluster import KMedoids

kmedoids = KMedoids(n_clusters=3, random_state=42)
clusters = kmedoids.fit_predict(X)

# Comparer les centres
print("Centres k-means:", kmeans.cluster_centers_)
print("Medoids (points réels):", kmedoids.cluster_centers_)
```

Exercice de réflexion : Dans quels cas préférer k-medoids à k-means ?

C) Classification hiérarchique - Voir la structure complète

Avantage : Pas besoin de fixer k à l'avance !

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Créer le dendrogramme
linkage_matrix = linkage(X, method='ward')

plt.figure(figsize=(12, 6))
dendrogram(linkage_matrix)
plt.title('Dendrogramme - Classification Hiérarchique')
```

Comment lire le dendrogramme :

- L'axe vertical montre la distance entre clusters
- Couper à différentes hauteurs donne différents nombres de groupes
- Les branches qui se rejoignent tard sont très différentes

D) Évaluer la qualité du clustering

Coefficient de silhouette (le plus intuitif) :

- Varie de -1 à +1
- Proche de +1 = excellent clustering
- Proche de 0 = clusters se chevauchent
- Négatif = mauvais clustering

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X, clusters)
print(f"Score de silhouette : {score:.3f}")
```

Tableau comparatif à créer :

Méthode	Nombre de clusters	Silhouette	Temps de calcul	Robustesse outliers
K-means	3	0.65	Rapide	Faible

Gamma fonctionne mieux sur Chrome.

Pour bénéficier d'une expérience optimale, ouvrez cette page dans Chrome.

Module 5 : Ajuster et diagnostiquer vos modèles

A) Validation croisée K-fold

Pourquoi ? Utiliser un seul découpage train/test peut être trompeur. La validation croisée donne une estimation plus fiable.

Le principe :

1. Diviser les données en K parties (ex: K=5)
2. Entraîner K fois : à chaque fois, une partie sert de test, les autres de train
3. Calculer la performance moyenne et son écart-type

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f"Accuracy moyenne : {scores.mean():.3f} (+/- {scores.std():.3f})")
```



Interprétation :

- Écart-type faible = modèle stable
- Écart-type élevé = modèle sensible aux données

B) Détecter le surapprentissage

Gamma fonctionne mieux sur Chrome.

Pour bénéficier d'une expérience optimale, ouvrez cette page dans Chrome.

```
train_sizes, train_scores, test_scores = learning_curve(  
    model, X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 10)  
)  
  
plt.plot(train_sizes, train_scores.mean(axis=1), label='Score train')  
plt.plot(train_sizes, test_scores.mean(axis=1), label='Score test')  
plt.xlabel('Nombre d\'exemples d\'entraînement')  
plt.ylabel('Score')  
plt.legend()  
plt.show()
```

Diagnostics visuels :

Surapprentissage

Grand écart entre courbe train (haute) et test (basse)

Solution : plus de données, régularisation, modèle plus simple

Sous-apprentissage

Les deux courbes basses

Solution : modèle plus complexe, plus de features

C) Optimiser les hyperparamètres

Grid Search - Recherche exhaustive :

```
from sklearn.model_selection import GridSearchCV  
  
param_grid = {
```


Gamma fonctionne mieux sur Chrome.

Pour bénéficier d'une expérience optimale, ouvrez cette page dans Chrome.

```
}

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='accuracy'
)

grid_search.fit(X_train, y_train)

print("Meilleurs paramètres :", grid_search.best_params_)
print("Meilleur score :", grid_search.best_score_)
```



Astuce : Commencez par une grille large, puis affinez autour des meilleures valeurs.

Module 6 : IA Responsable - Identifier et corriger les biais

Pourquoi c'est crucial : Un modèle biaisé peut discriminer des groupes et avoir des conséquences graves (refus de crédit, recrutement, justice).

A) Identifier les biais

Étape 1 : Repérer les attributs sensibles

- Sexe, âge, origine ethnique, code postal, etc.
- Même si vous ne les utilisez pas directement, d'autres variables peuvent être des proxys

Étape 2 : Mesurer l'équité

```
# Exemple : prédiction d'octroi de crédit
from sklearn.metrics import confusion_matrix

# Performance globale
```

```
# Performance par groupe
for group in ['homme', 'femme']:
    mask = (X_test['sexe'] == group)
    accuracy_group = accuracy_score(y_test[mask], y_pred[mask])
    print(f"Accuracy {group}: {accuracy_group:.3f}")
```

Métriques d'équité (Google Fairness) :

1. Parité démographique

Les groupes reçoivent-ils des prédictions positives à taux égal ?

```
taux_positif_groupe_A = (y_pred[groupe_A] == 1).mean()
taux_positif_groupe_B = (y_pred[groupe_B] == 1).mean()
ecart = abs(taux_positif_groupe_A - taux_positif_groupe_B)
```

2. Égalité des chances

Parmi les cas positifs réels, les groupes sont-ils détectés équitablement ?

```
tpr_A = recall_score(y_test[groupe_A], y_pred[groupe_A])
tpr_B = recall_score(y_test[groupe_B], y_pred[groupe_B])
```

B) Techniques de mitigation



Option 1 : Rééquilibrer les données



Option 2 : Repondérer les classes



Option 3 : Supprimer les variables sensibles
(avec précaution !)

```
# Sur-échantillonner la classe minoritaire
smote = SMOTE(random_state=42)
X_resampled, y_resampled =
smote.fit_resample(X_train,
y_train)
```

```
'balanced')
```

⚠ **Dilemme éthique** : Parfois, équité et performance sont en tension. Comment arbitrer ?

C) Interprétabilité avec SHAP

Pourquoi ? Comprendre quelles variables influencent les prédictions.

```
import shap

# Créer l'explicateur
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)

# Visualisation globale
shap.summary_plot(shap_values, X_test)

# Explication locale (un individu)
shap.waterfall_plot(shap_values[0])
```

Interprétation :

- **Rouge** = augmente la prédiction positive

Questions à se poser :

- Les variables importantes ont-elles du sens métier ?
- Le modèle s'appuie-t-il sur des corrélations légitimes ?
- Y a-t-il des variables sensibles qui influencent trop ?

D) Recommandations pour un usage responsable

Checklist à suivre :

1

Données représentatives

Tous les groupes sont-ils bien représentés ?

2

Documentation

Créer une "datasheet" du modèle (architecture, données, limites)

3

Monitoring

Surveiller les performances dans le temps (drift des données)

4

Tests d'équité

Évaluer régulièrement les métriques de fairness

5

Processus de recours

Permettre la contestation des décisions automatisées



Principe clé : Un modèle n'est jamais "terminé" – il nécessite une surveillance continue.

Partie III – Machine Learning Supervisé Avancé

Module 7 : Approche par classe inversée

Format : Vous posez vos questions, nous les résolvons ensemble !

Prérequis à maîtriser :

Concept	Définition simple	Pourquoi c'est important
Variable explicative (X)	Les données d'entrée utilisées pour prédire	Ce sont vos "indices" pour faire la prédiction
Variable cible (y)	Ce qu'on cherche à prédire	L'objectif final de votre modèle
Apprentissage supervisé	Apprendre à partir d'exemples étiquetés	Le modèle apprend la relation $X \rightarrow y$
Validation croisée	Tester le modèle sur plusieurs découpages	Garantir que les résultats ne sont pas dus au hasard
Hyperparamètres	Paramètres qu'on fixe avant l'entraînement	Ils contrôlent la complexité du modèle

Module 8 : Méthodes de classification avancées

A) Arbres de décision

Pourquoi c'est puissant : Très interprétables, gèrent les non-linéarités.

Comprendre la construction :

1. **À chaque nœud, choisir la meilleure question :**
 - "Âge > 30 ans ?" ou "Revenu > 50k€ ?"
 - Critère : maximiser le gain d'information (ou minimiser l'impureté de Gini)
2. **Continuer jusqu'à une condition d'arrêt :**
 - Profondeur maximale atteinte
 - Nombre minimal d'exemples par feuille
 - Gain d'information insuffisant

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_train)

# Visualiser l'arbre
plt.figure(figsize=(20,10))
plot_tree(tree, feature_names=X.columns, class_names=['Non', 'Oui'], filled=True)
plt.show()
```

📄 💡 **Exercice :** Lisez l'arbre et trouvez les règles de décision :

- Exemple : "Si âge ≤ 30 ET revenu > 40k ALORS prédiction = Oui"

⚠️ **Risque :** Les arbres profonds surajustent facilement. Limitez max_depth !

B) Random Forest - La puissance du collectif

Gamma fonctionne mieux sur Chrome.

Pour bénéficier d'une expérience optimale, ouvrez cette page dans Chrome.

1. Bagging

Chaque arbre est entraîné sur un échantillon bootstrap (tirage avec remise)

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100, # Nombre d'arbres
    max_depth=10,
    min_samples_split=20,
    random_state=42
)
rf.fit(X_train, y_train)

# Importance des variables
importances = pd.DataFrame({
    'feature': X.columns,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)

print(importances)
```

Avantages

- ✓ Réduit le surapprentissage (variance plus faible qu'un arbre seul)
- ✓ Gère bien les données complexes et non-linéaires
- ✓ Peu sensible aux hyperparamètres

2. Feature randomness

À chaque nœud, considérer seulement un sous-ensemble aléatoire de variables

Inconvénient

Moins interprétable qu'un arbre unique.

C) Boosting - Apprendre de ses erreurs

Principe : Entraîner des modèles séquentiellement, chaque nouveau modèle se concentrant sur les erreurs du précédent.

Gradient Boosting (le plus utilisé) :

```
n_estimators=100,
learning_rate=0.1,
max_depth=3,
random_state=42
)
gb.fit(X_train, y_train)

print(f"Accuracy train: {gb.score(X_train, y_train):.3f}")
print(f"Accuracy test: {gb.score(X_test, y_test):.3f}")
```

Hyperparamètres clés :

- **learning_rate** : pas d'apprentissage (typiquement 0.01-0.3)
 - Plus petit = convergence plus lente mais souvent meilleure
- **n_estimators** : nombre d'arbres
 - Plus grand = plus puissant mais risque de surapprentissage
- **max_depth** : complexité de chaque arbre (typiquement 3-8)

⚠ **Attention** : Le boosting peut facilement surajuster. Surveillez la différence train/test !

D) Tableau comparatif - Synthèse pratique

Exercice final : Sur votre jeu de données, comparez tous les modèles :

Modèle	Accuracy	Précision	Rappel	F1-Score	Temps d'entraînement	Interprétabilité
KNN (k=5)	0.82	0.79	0.85	0.82	2s	Faible
Arbre (depth=4)	0.84	0.83	0.82	0.82	1s	Excellente

Gamma fonctionne mieux sur Chrome.

Pour bénéficier d'une expérience optimale, ouvrez cette page dans Chrome.

	Gradient	0.90	0.89	0.91	0.90	403	Faible
Boosting							

Questions de réflexion :

1. Quel est le meilleur modèle selon vos critères ?
2. Sacrifieriez-vous 2% de performance pour une meilleure interprétabilité ?
3. Le temps de calcul est-il acceptable pour votre cas d'usage ?